Quiero que generes una **aplicación web full-stack, mobile-first** , para registrar y analizar jornadas de venta y distribución de alimentos realizadas en los diferentes Estados de Venezuela. La aplicación será utilizada por **Inspectores**, **Jefes** y **Administradores**, cada uno con permisos y pantallas específicas.

Objetivo general

La aplicación debe permitir que los Inspectores registren **reportes de actividad** (con datos de alimentos, cantidades, número de personas atendidas, ubicación geográfica automática, etc), y que los Jefes y Administradores consulten y analicen esta información en un tablero analítico con mapas y gráficos. Además, debe existir un sistema de auditoría robusto para todas las acciones del rol Administrador.

Para todos los efectos que necesites desarrollar en Supabase utiliza @mcp:supabase-mcp-server: (Yo dispongo los datos del proyecto creado; url – anon public, etc, solicítame estos datos cuando lo vayas a utilizar.

Para totos los efectos del diseño y experiencia UX/UI utiliza @mcp:stitch:

En el Desarrollo de la Aplicación vas a necesitar unos datos para completar la información en determinadas listas desplegables tales como: Estados – Rubros – Empaque – Medida, etc etc, esta información debes solicitarla cuando la requieras para proporcionártela.

Roles y funciones del sistema

El usuario rol de **Inspector**:

- Debe poder **registrarse** en el sistema mediante un formulario de alta de usuario (nombre, apellido, cédula, correo, contraseña (pin 6 dígitos), Estado y teléfono).
- El usuario en el Rol de Inspector es el único que debe Registrarse y Logearse.
- Debe poder **autenticarse** (iniciar sesión) con credenciales únicas (cédula + contraseña).
- Debe poder crear un **nuevo informe de actividad** a través de un formulario estructurado (módulo “Elaborar Reporte”).
- Debe poder **guardar un informe en estado “borrador”** y editarlo libremente mientras mantenga ese estado (para corregir errores antes del envío).
- Debe poder **enviar un informe** cambiando su estado a “enviado”; A partir de ese momento, el informe queda **bloqueado para edición** (solo lectura para el Inspector).
- Debe tener acceso de **solo lectura** a una **tabla resumen de todos los informes del día** (solo los suyos):
    - La tabla debe mostrar como mínimo las columnas: Fecha, Hora, Tipo de actividad y Lugar (punto de atención).
    - Permitir ordenar y filtrar por fecha y tipo de actividad.

Rol Jefe

El usuario rol **Jefe**:

- Debe tener acceso a un **módulo de tablero analítico** (dashboard).
- El usuario con el Rol de **Jefe;** no requiere “Registrarse”, solo se Loguea usando una clave única para Cédula: “JEFE” y el Pin será: 123456.
- El tablero debe incluir, como mínimo:
    1. Filtros por diferentes categorías y los gráficos deben actualizarse en función del filtro aplicado.
    2. **Gráfico lineal de actividades por fecha**
        - Eje X: fechas de las actividades (por día).
        - Eje Y: número de actividades registradas en cada fecha.

3. **Gráfico de barras agrupadas por Estado**
    - Eje X: Estados (regiones del país).
    - Eje Y: número de actividades realizadas en cada Estado.
4. **Mapa de Venezuela con marcadores georreferenciados**
    - Mapa centrado en Venezuela.
    - Marcadores (pins) en los puntos donde se realizan las actividades, usando la latitud y longitud capturadas (por georreferencia) en los informes.
    - Al hacer clic en un pin, se mostrará una pequeña ventana emergente con información básica: fecha, inspector, tipo de actividad, lugar.
5. Gráfico de Pie para indicar en porcentaje (%) el total (tn) de alimentos a distribuir:
    - Frutas
    - Verduras
    - Hortalizas
    - Proteínas
    - Secos
6. **Gráficos de barras de cobertura social**
    - Un gráfico de barras que muestra la **cantidad de comunas atendidas** (por Estado o nivel nacional, indica claramente).
    - Un gráfico de barras que muestra la **cantidad de familias atendidas** (por Estado o nivel nacional, indica claramente).
7. **Gráfico de barras de Inspectores activos por Estado**
    - Definir "inspector activo" como aquel que ha generado al menos un informe en un rango de fechas configurable (por defecto, últimos 30 días).
    - Eje X: Estados.
    - Eje Y: número de inspectores activos.
8. **Indicadores binarios por informe (etiquetas "Sí/No")**

Para cada informe, deben existir campos que indiquen, en formato de etiquetas o insignias, las siguientes condiciones sacadas de la suma de los Sí o los No:

- Variedad de rubros (Sí/No).
- Bodega móvil limpia (Sí/No).
- Personal suficiente (Sí/No).
- Comunidad notificada (Sí/No).
- Presencia de contaminación en la zona (Sí/No).
- Presencia de proteínas (Sí/No).
- Disponibilidad de espacio de almacenamiento en frío (Sí/No).
- Presencia de frutas (Sí/No).
- Presencia de verduras (Sí/No).
- Presencia de hortalizas (Sí/No).

Estos campos deben poder visualizarse tanto en el detalle del informe como en alguna vista resumida o tabla (por ejemplo, con iconos o etiquetas de color).

9. **Gráfico sobre métodos de pago**
    - Debe incluir un gráfico (barras, torta o el que considere más claro) que muestre la **distribución de los métodos de pago** reportados en las actividades (por ejemplo: efectivo, punto de venta, transferencia, pago móvil, otros).

- El tablero debe permitir filtrar por rango de fechas, Estado, tipo de actividad y/o inspector.
- La app debe permitir generar archivos en formato "PDF" para descargar

---

Rol Administrador

El usuario rol **Administrador**:

- El usuario con el Rol de **Administrador;** no requiere "Registrarse", solo se Loguea usando una clave única para Cédula: "ADMIN" y el Pin será: 654321.
- Debe poder **crear nuevos Inspectores** (alta de usuario Inspector) mediante un formulario administrativo.
- Debe poder **eliminar registros de Inspectores** (baja lógica preferiblemente, evitando borrados físicos de datos, salvo que se especifique lo contrario).
- Debe poder acceder al **mismo panel de visualización (dashboard) que el rol Jefe**, con los mismos gráficos y filtros.
- Debe poder **agregar, eliminar o modificar campos del formulario de reporte** (gestión dinámica de campos), incluyendo:
    - Campos de texto, numéricos, booleanos (Sí/No), listas desplegables.
    - Posibilidad de marcar campos como obligatorios u opcionales.
    - Posibilidad de reordenar los campos dentro de secciones (por ejemplo, "Datos generales", "Alimentos", "Condiciones del entorno").

Auditoría (bitácora de operaciones del Administrador)

- El sistema debe registrar en una **bitácora de auditoría** todas las operaciones realizadas por el usuario con rol Administrador sobre:
    - Usuarios (crear, modificar, eliminar).
    - Configuración del sistema.
    - Formularios de informe (definición de campos, cambios en estructura, etc.).
- Cada entrada de auditoría debe incluir al menos:
    - Identificador del administrador (ID de usuario).
    - Fecha y hora exacta de la operación (con zona horaria adecuada).
    - Tipo de operación (crear, modificar, eliminar, activar/desactivar, etc.).
    - Entidad afectada (por ejemplo: "Inspector", "Campo de formulario", "Configuración").
    - Valores anteriores y nuevos (cuando se aplican), por ejemplo en formato JSON o estructura clara.
- La bitácora de auditoría debe ser **inalterable** (modelo de almacenamiento solo de anexado / *append-only*):
    - Ningún, incluido el Administrador de la aplicación o de la base de datos de usuario, podrá **modificar ni eliminar** registros una vez escritos.
    - Solo se permitirán nuevas entradas (agregar registros).
    - Implementar este comportamiento a nivel de modelo y, si es posible, con mecanismos adicionales (por ejemplo, restricciones de base de datos o disparadores) que lo refuercen.
- El acceso a la bitácora de auditoría se limitará a **perfiles autorizados de supervisión o cumplimiento**, únicamente en **modo lectura**:
    - Acceso mediante consultas o informes predefinidos.
    - No debe haber ninguna opción en la interfaz para editar o eliminar entradas de auditoría.
- Cualquier intento de:
    - cambios de registros de auditoría,
    - desactivación del mecanismo de auditoría, o
    - fallo del mecanismo de auditoría,

debe generar automáticamente:

    - Un **evento de alerta** interna.
    - Un **registro adicional en la misma bitácora** describiendo el incidente (fecha/hora, usuario implicado si lo hay, tipo de intento o fallo, detalles técnicos relevantes).

Funcionalidades principales del formulario de reporte (Inspector)
El formulario de "Elaborar Reporte" debe incluir, como mínimo:

- Sección "Datos Generales":
    - Nombre Inspector (tomado automáticamente del usuario logueado).
    - Fecha y hora de la actividad (por defecto, ahora; no editable).
    - Tipo de actividad: "FCS", "FCS - Emblemática", "Bodega móvil".
    - Empresa: (Lista anexa accionada mediante desglosable para seleccionar).
    - Estado. (Lista anexa accionada mediante desglosable para seleccionar).
    - Municipio: (Texto máx. 50 caracteres).
    - Parroquia: (Texto máx. 50 caracteres).
    - Número de **comunas atendidas** (entero ≥ 0).
    - Número de **familias atendidas** (entero ≥ 0).
    - Número total de **personas atendidas** (entero > 0).
    - Georreferenciación automática (latitud – longitud mediante GPS del dispositivo móvil)
- Sección "Contactos":
    - Responsable Actividad
        - Nombre y Apellido: (Texto máx. 50 caracteres).
        - Cédula: (Texto máx. 15 caracteres).
        - Teléfono: (Texto máx. 15 caracteres).
    - Responsable Enlace Comuna
        - Nombre y Apellido: (Texto máx. 50 caracteres).
        - Cédula: (Texto máx. 15 caracteres).
        - Teléfono: (Texto máx. 15 caracteres).
- Sección "Condiciones e indicadores binarios":
    - Campos Sí/No para:
        - Variedad de rubros.
        - Bodega móvil limpia.
        - Personal suficiente.
        - Comunidad notificada.
        - Presencia de contaminación en la zona.
        - Presencia de proteínas.
        - Disponibilidad de espacio de almacenamiento en frío.
        - Presencia de frutas.
        - Presencia de verduras.
        - Presencia de hortalizas.
- Métodos de pago:
    - Uno o varios métodos selección múltiple (efectivo [moneda nacional], efectivo [divisa], punto de venta, transferencia, pago móvil, plataforma externa [zelle, Binace, etc], No aplica).
- Sección "Rubros alimenticios".
    - Lista dinámica de artículos de alimento, con:
        - Rubro (catálogo o texto que será suministrado).
        - Empaque: (texto manual libre / máx 30 caracteres).

        - Medida (kg, grs, raciones, combos, litros, mil, otro).
        - Precio (Bs): (texto manual libre / máx 15 caracteres).
        - Cantidad total (tn) para distribuir: (número positivo >= 1).
        - Observaciones: (texto manual libre / máx 220 caracteres, incluir contador para verificar espacio disponible).
- Sección “Registro Fotográfico”
    - La Aplicación debe permitir subir y enviar hasta un máximo de 3 fotografías.

Georreferenciación automática

- Al abrir el formulario para crear un informe, la aplicación debe solicitar permiso de ubicación al navegador (API de geolocalización).
- Debe capturar la latitud y longitud del dispositivo y mostrarlas en los campos correspondiente de solo lectura.
- Es obligatorio que existan coordenadas válidas para poder enviar el informe.
    - Si el usuario no da permiso o la ubicación falla, muestre mensajes claros y permita reintentar la obtención.
    - Bloquear el botón de envío mientras no haya coordenadas válidas.
- Estas coordenadas se guardarán con el informe y se usarán para mostrar marcadores en el mapa del tablero.

Requisitos técnicos

- **Interfaz**:
    - React con TypeScript.
    - Diseño **mobile-first** y totalmente responsivo (100%).
    - Componentes para:
        - Iniciar sesión/Registrarse.
        - Formulario de informe (Inspector).
        - Listado de reportes del día (Inspector).
        - Panel de control (Jefe/Administrador) con gráficos y mapas.
        - Pantallas de administración de usuarios y campos dinámicos (Administrador).
- **Backend** :
    - Node.js con Express (o framework similar).
    - Autenticación con JWT.
    - Middleware de control de acceso por rol (Inspector, Jefe, Administrador, y, si se define, Perfil de supervisión/auditoría).
    - Puntos finales para:
        - Autenticación y gestión de usuarios.
        - CRUD de informes con estados (borrador, enviado).
        - Consulta agregada para el tablero (datos para gráficos y mapa).
        - Gestión de campos dinámicos de formularios.
        - Registro y consulta (solo lectura) de la bitácora de auditoría.
- **Base de datos**:
    - SQL (por ejemplo, PostgreSQL).
    - Tablas mínimas:
        - users(con rol, datos básicos, estado, etc.).
        - reports(datos generales, coordenadas, estado del informe, vínculos a usuario y ubicación).

        - report_items(lista de alimentos).
        - audit_log(bitácora de auditoría en modelo append-only).
        - Tablas necesarias para todos los campos dinámicos de formularios (por ejemplo, form_fields, form_field_options, etc.).
    - DURANTE LA FASE DE DESARROLLO SE USARÁ **SUPABASE** LUEGO SE MIGRARÁ LA APP PARA LOS SERVIDORES PROPIOS.

Experiencia de usuario/interfaz de usuario

- Interfaz en español, simple y clara, pensada para uso en campo (sol, ruido, conexiones inestables).
- Formularios de una sola columna en móvil, con botones grandes y mensajes de error claros.
- En el tablero, gráficos y mapas bien organizados, con filtros visibles.
- No cambiar la lógica de negocio si se optimiza la estética.

Entregables importantes que debes cumplir:

- Código completo de frontend y backend.
- Esquema o migraciones de base de datos.
- Implementación de roles y permisos.
- Implementación de bitácora de auditoría append-only.
- Instrucciones básicas de ejecución y configuración.
- Archivo .APK para descargar la App para usarla en el terreno y el enlace para la WEB para uso del Jefe y el Administrador. Solo la entregaras cuando el Desarrollar haya completado la App, este satisfecha y te lo solicite.

Si algún detalle razonable falta para que el sistema funcione de extremo a extremo, complétalo de forma coherente con esta descripción, manteniendo siempre las restricciones de roles, auditoría y muy importante la georreferenciación.

Por último antes de desarrollar la APP puedes hacer todas las preguntas necesarias para que tu estes claro con lo que necesito de manera de generar un concepto bien Robusto, Elegante, Útil y de fácil uso para el usuario.

MENSAJES PARA AJUSTAR LA APP

1. **Mejorar el mapa y la georreferenciación**
   - Centra el mapa automáticamente en la latitud y longitud del informe con un zoom adecuado para ver el detalle barrio.
   - Muestra una información o ventana emergente sobre el marcador (pin) con: nombre del punto de atención, tipo de actividad y fecha.
   - Si por algún motivo el reporte no tiene coordenadas, muestra un mensaje en lugar del mapa ('Este reporte no tiene ubicación registrada').”
   - “Optimiza el uso del mapa en la pantalla de detalle del informe:
   - Asegúrese de que el mapa solo se inicialice una vez por montaje del componente, no en cada render.
   - Extrae el mapa a un componente separado y méte la inicialización en un efecto (useEffect) bien controlado.
   - Si el mapa carga recursos pesados, evalúa cargar la librería de mapas de forma perezosa (lazy) solo cuando se abre el detalle.
   - Revisa que no se hagan llamadas innecesarias a la API de mapas en cada cambio de estado del componente.”
   - Formularios mucho más utilizables en móvil
   - “Sobre la aplicación actual, mejora la UX/UI del formulario de reporte siguiendo buenas prácticas de formularios móviles:
   - Usa una sola columna con campos grandes y fáciles de tocar, con suficiente espacio entre ellos.
   - Agrupa campos relacionados en bloques visuales (Datos de la actividad, Alimentos, Ubicación).
   - Active el foco automáticamente en el primer campo y resalte claramente el campo activo.
   - Usa teclado numérico para campos de números y limita la longitud visible de cada campo según el tipo de dato.
   - Valida los campos uno por uno al salir de cada campo y muestra el error inmediatamente debajo, sin esperar al envío.”
2. **Agregar mejoras visuales y de estilo**
   - Usa una tipografía tipo system font stack (ej. system-ui, -apple-system, BlinkMacSystemFont).
   - Mantén un diseño minimalista, sin animaciones excesivas, siempre mobile-first.”
   - Agregue un encabezado fijo con el título de la aplicación ('Reportes de FCA') y un botón para cerrar sesión.
   - Mejora la tabla de listados con filas alternas ligeramente sombreadas y efecto de desplazamiento.
3. **Revisar calidad del código y rendimiento**
   - Identifique los componentes React que se puedan extraer para reducir la duplicación.
   - Sugiera dónde usar memorización para evitar resultados innecesarios.
   - Revise las rutas y controladores de Express para eliminar código repetido y mejorar el manejo de errores.
   - Asegúrate de que todas las llamadas a la API manejen correctamente estados de carga y error en el frontend.
   - Haz los cambios directamente en el código y explica brevemente en comentarios qué se mejoró.”

4. **Optimizar el frontend (React)**
   - “Sobre la app actual, revisa el frontend en React y optimiza rendimiento y fluidez:
   - Identifica los componentes que se están rerenderizando sin necesidad y usa React.memo o separación de componentes para reducir los renders.
   - Asegúrese de que las listas de informes sean keyestables y eficientes.
   - Implementa carga perezosa (carga diferida) para pantallas que no se usan al inicio, como el detalle del informe o la sección de ayuda.
   - Revise el manejo de estado: evita guardar en estado global datos que solo necesita un componente.
5. **Paginación y carga eficiente de informes**
   - Implementa paginación en el backend (limit, offseto page, pageSize) y en el frontend.
   - Carga solo la página actual de informes en lugar de traer todos de golpe.
   - Agregue controles de paginación (Siguiente, Anterior, ir a página) y muestra cuántos resultados totales hay.
   - Asegúrese de que los filtros (rango de fechas, tipo de actividad, búsqueda por texto) funcionen bien con la paginación.
   - Minimiza el tamaño de la respuesta del backend devolviendo solo los campos necesarios en el listado.”
6. **Manejo eficiente de errores y tiempos de espera**
   - “Optimiza el manejo de errores y tiempos de espera para mejorar la sensación de rendimiento:
   - Configure tiempos de espera razonables para las solicitudes HTTP en el frontend.
   - Muestra estados de carga ligeros ('Cargando...') en componentes específicos en lugar de bloquear toda la pantalla.
   - Implementa reintentos (retry) controlados para las llamadas críticas, como la creación de informes, con mensajes claros al usuario.
   - Asegúrate de que, si falla la geolocalización, no se intente llamar al backend innecesariamente.”
7. **Formularios mucho más utilizables en móvil**
   - “Sobre la aplicación actual, mejora la UX/UI del formulario de reporte siguiendo buenas prácticas de formularios móviles:
   - Usa una sola columna con campos grandes y fáciles de tocar, con suficiente espacio entre ellos.
   - Active el foco automáticamente en el primer campo y resalte claramente el campo activo.
   - Usa teclado numérico para campos de números y limita la longitud visible de cada campo según el tipo de dato.
   - Valida los campos uno por uno al salir de cada campo y muestra el error inmediatamente debajo, sin esperar al envío.”
8. **Prototipo de una versión “ultra rápida” del flujo principal**
   - Usa valores por defecto inteligentes (fecha/hora actual, ubicación, responsable desde la sesión, tipo de actividad preseleccionado si suele repetirse).
   - Permite agregar un solo alimento rápidamente y, opcionalmente, más elementos con un botón 'Agregar otro alimento'.

9. **Requisitos de diseño responsivo:**
   - **Móvil primero:**
     - Priorice las pantallas del móvil (320–414 px de ancho).
     - Formularios en una sola columna, campos y botones grandes, con suficiente espacio para usar con el pulgar.
     - Menú de navegación simplificado en la parte superior con ícono tipo 'hamburguesa' si hace falta.
   - **Comportamiento por tamaño de pantalla:**
     - En móvil: una sola columna, la lista de informes ocupa todo el ancho, tarjeta por informe apilada.
     - En tableta: dos columnas para la lista de informes (dos tarjetas por fila) manteniendo el formulario en una sola columna.
   - **En el escritorio:**
     - Diseño de dos paneles: listado de informes a la izquierda y detalle del informe (con mapa) a la derecha.
     - Usa un ancho máximo legible (por ejemplo 1200 px) y centra el contenido.
   - **Buenas prácticas de responsive:**
     - Usa unidades relativas (%, rem, vh, vw) en lugar de depender solo de px.
     - Asegúrese de que el texto y los botones tengan buen contraste y tamaño mínimo legible en el móvil.
     - El mapa debe adaptarse al ancho disponible en cada dispositivo sin desbordar el contenedor.
     - Implementa media queries claras para los puntos de interrupción de móvil, tableta y escritorio.
   - **Experiencia de usuario/interfaz de usuario:**
     - Mantén la jerarquía visual clara: título de pantalla, secciones ('Datos de la actividad', 'Alimentos', 'Ubicación, etc') bien separadas con espacio.
     - En el móvil, evita menús laterales complicados; prioriza la navegación sencilla: Listado → Detalle → Volver.
     - Asegúrese de que no haya barras de desplazamiento horizontales en ningún dispositivo.
     - Actualiza los estilos (CSS/Tailwind/lo que uses), los componentes y el diseño para cumplir todo lo anterior, sin cambiar la lógica de negocio ni la API.”